COSMIC UNIVERSALISM COMPUTATIONAL INTELLIGENCE INITIATIVE

Cosmic Breath Time Converter

Implementation Manual

A specification-first, controlled migration manual for preserving the approved converter behavior while integrating a testable TypeScript engine and accessible interactive interface into the Astro CU-Time page.

Repository	CosmicUniversalismStatement
Protected branch	`main`
Stable website branch	`website`
Active feature branch	`feature/cu-time-converter`
Foundation tag	`website-v1.1-cu-time-foundation`
Foundation commit	`4fd6897`
Target route	`/CosmicUniversalismStatement/cu-time/`
Migration strategy	Approach B: tested TypeScript extraction
Reference implementation	`cosmic_breath_time_converter_v3_0_1.html`
Document version	1.0
Verified	July 19, 2026

William Maddock

https://github.com/willmaddock/CosmicUniversalismStatement

Document Control

Document	Cosmic Breath Time Converter: Implementation Manual
Version	1.0
Classification	Technical Specification and Controlled Migration Manual
Repository	CosmicUniversalismStatement
Protected branch	`main`
Stable integration branch	`website`
Active development branch	`feature/cu-time-converter`
Foundation baseline	`website-v1.1-cu-time-foundation` at `4fd6897`
Target page	`website/src/pages/cu-time.astro`
Target local route	`/CosmicUniversalismStatement/cu-time/`
Reference implementation	`cosmic_breath_time_converter_v3_0_1.html`
Technical baseline verified	July 19, 2026

Purpose

This manual controls the solo-workflow migration of the existing standalone Cosmic Breath Time Converter into the Cosmic Universalism Astro website. It preserves approved numerical behavior while separating mathematical specification, pure conversion logic, browser interaction, presentation, testing, and release approval.

Safety Rule

This document does not approve any mathematical correction merely because a potential defect or ambiguity is identified. The original HTML converter is the behavioral reference. Every intentional change to constants, formulas, rounding, calendar interpretation, labels, or outputs requires a documented review decision and new test expectations.

Contents

START HERE - Initialize a New Project Chat

Use this section whenever converter work is continued in a new chat inside the existing **Cosmic Universalism Website** ChatGPT Project.

Required Project references

Add or retain these files in the Project:

- `Cosmic_Universalism_Website_Implementation_Manual_v1.1.pdf`
- `Cosmic_Breath_Time_Converter_Implementation_Manual_v1.0.pdf`
- `cosmic-universalism-website-blueprint.md`
- `cosmic_breath_time_converter_v3_0_1.html`

New-chat bootstrap prompt

Copy-and-Paste Prompt: Initialize the CU-Time Converter Chat

```
We are continuing the controlled Cosmic Breath Time Converter
implementation inside the Cosmic Universalism Website Project.

Before proposing or making changes, read these Project references:

1. Cosmic_Breath_Time_Converter_Implementation_Manual_v1.0.pdf
2. Cosmic_Universalism_Website_Implementation_Manual_v1.1.pdf
3. cosmic_breath_time_converter_v3_0_1.html
4. cosmic-universalism-website-blueprint.md

Use the Project Instructions as mandatory governance rules.

Repository:
willmaddock/CosmicUniversalismStatement

Protected branch:
main

Stable website branch:
website

Active feature branch:
feature/cu-time-converter

Foundation baseline tag:
website-v1.1-cu-time-foundation

Foundation baseline commit:
4fd6897
```

```
Target page:
website/src/pages/cu-time.astro

Target route:
/CosmicUniversalismStatement/cu-time/

Reference implementation:
cosmic_breath_time_converter_v3_0_1.html

Selected migration strategy:
Approach B - preserve approved converter behavior while extracting
its mathematics into testable TypeScript modules and connecting an
accessible interface to the Astro CU-Time page.

The original HTML converter must remain unchanged as the reference
implementation.

First:

1. Summarize the applicable Project protection rules.
2. Summarize the verified foundation baseline.
3. Identify the current authorized manual phase.
4. Identify the target route and expected CU-Time files.
5. State which repository facts must be verified before editing.
6. State the next authorized task and its completion gate.

Do not edit files.
Do not install packages.
Do not switch branches.
Do not commit, tag, push, merge, or deploy.
Wait for repository verification before implementation.
```

Checkpoint

A new chat is initialized correctly only when it identifies `main` as protected, `website` as the stable baseline, `feature/cu-time-converter` as the active feature branch, and the original HTML converter as an unchanged reference implementation.

1 Part 1 - Project Governance and Verified Baseline

1.1 Repository and branch authority

Item	Verified state
Repository root	`/Users/cosmos/Documents/GitHub/CosmicUniversalismStatement`
Origin	https://github.com/willmaddock/CosmicUniversalismStatement.git
Protected branch	`main`; never modify for this workflow
Stable website branch	`website` at `a4a4666`
Stable website tag	`website-v1.0-foundation`
Active feature branch	`feature/cu-time-converter`
Feature foundation commit	`4fd68974e6a8ff99289087cb195a7b09f5759386`
Feature foundation tag	`website-v1.1-cu-time-foundation`
Working tree	Clean and tracking `origin/feature/cu-time-converter`
Deployment trigger	`website` only

1.2 Verified technology state

Technology	Verified state
Node.js	`v24.18.0`
npm	`11.16.0`
Astro	`^7.1.1`
TypeScript configuration	Astro strict configuration
Astro site	`https://willmaddock.github.io`
Astro base	`/CosmicUniversalismStatement`
Arbitrary-precision dependency	Not installed
Automated test framework	Not installed
Existing client scripts	None detected in `website/src/`

1.3 Verified foundation files

Existing CU-Time files

```
website/src/pages/cu-time.astro
website/src/components/CUTimeConverter.astro
website/src/components/CUTimeInvitation.astro
website/src/lib/cuTime/constants.ts
```

The current page is explanatory and imports the foundation component. The component contains accessible presentation markup, classification, and an active-development status. It does not contain controls or calculations.

The constants workspace is not empty. It contains an initial inventory of NASA-aligned and CU framework values as ordinary JavaScript number literals. Those values are not yet approved for calculation and may not preserve all source digits when evaluated as binary floating-point numbers.

Do Not Continue Until Resolved

Do not describe `constants.ts` as an empty placeholder. The accurate status is: **inventory extracted, precision representation and mathematical approval pending**.

1.4 Verified production build

The foundation build was executed on July 19, 2026.

Verified build result

```
Command: npm run build
Exit code: 0
Output mode: static
Pages built: 7
CU-Time route generated: /cu-time/index.html
Active branch: feature/cu-time-converter
HEAD: 4fd6897
Working tree after build: clean
```

Successful Result

The CU-Time presentation foundation is buildable and the target route exists. This authorizes specification work. It does not yet authorize mathematical integration.

1.5 Project Instructions appendix block

Append this block to the existing ChatGPT Project Instructions. Do not replace the original website instructions.

Copy-and-Paste Prompt: Permanent CU-Time Project Instructions

```
CURRENT CU-TIME DEVELOPMENT WORKFLOW

ACTIVE FEATURE:
Cosmic Breath Time Converter integration

STABLE WEBSITE BRANCH:
website

STABLE WEBSITE MILESTONE:
website-v1.0-foundation

STABLE WEBSITE COMMIT:
a4a4666

ACTIVE DEVELOPMENT BRANCH:
feature/cu-time-converter

CURRENT FEATURE FOUNDATION:
website-v1.1-cu-time-foundation
```

```
FOUNDATION COMMIT:
4fd6897

TARGET PAGE:
website/src/pages/cu-time.astro

TARGET LOCAL ROUTE:
/CosmicUniversalismStatement/cu-time/

CURRENT FOUNDATION COMPONENT:
website/src/components/CUTimeConverter.astro

CURRENT CONSTANTS INVENTORY:
website/src/lib/cuTime/constants.ts

REFERENCE CONVERTER:
cosmic_breath_time_converter_v3_0_1.html

IMPLEMENTATION APPROACH:
Approach B - extract approved converter behavior into pure,
testable TypeScript modules and integrate it with the Astro CU-Time
page using lightweight browser JavaScript.

FUNCTIONAL PARITY RULE:
The integrated converter must preserve approved numerical behavior,
precision, inputs, outputs, calendar handling, and conversion
capabilities of the reference converter. The website interface does
not need to preserve the reference converter's React, CDN, Tailwind,
or visual architecture.

REFERENCE PROTECTION:
Do not modify the original HTML reference converter during the
initial migration.

MATHEMATICAL PROTECTION:
1. Do not silently change constants.
2. Do not silently change formulas.
3. Do not silently change rounding behavior.
4. Do not silently change calendar assumptions.
5. Record each proposed correction separately.
6. Establish golden reference vectors before implementing the new
   conversion engine.
7. Demonstrate parity before introducing enhancements.

BRANCH PROTECTION:
1. Never modify main.
2. Keep website as the stable website baseline.
3. Perform CU-Time work only on feature/cu-time-converter.
4. Confirm branch and working-tree status before every edit.
5. Do not merge into website without explicit approval.
```

```
6. Do not commit, tag, push, merge, or deploy unless explicitly
   requested.

CURRENT DEVELOPMENT BOUNDARY:
The presentation foundation is complete. Constants review,
mathematical specification, engine implementation, automated tests,
and interactive integration remain pending.
```

1.6 Required pre-edit terminal check

Verify the live repository before every editing session

```
cd "$(git rev-parse --show-toplevel)" || exit 1

git branch --show-current
git rev-parse --short HEAD
git status --short --branch
```

Expected branch:

Required branch

```
feature/cu-time-converter
```

1.7 No-edit Codex inspection prompt

Copy-and-Paste Prompt: Verify the Live Repository

```
Inspect the current repository without changing any files.

Expected active branch:
feature/cu-time-converter

Foundation baseline tag:
website-v1.1-cu-time-foundation

Foundation baseline commit:
4fd6897

Verify and report:

1. Repository root and origin
2. Active branch and HEAD
3. Working-tree status
4. Relevant website and CU-Time tags
5. CU-Time-related file structure
6. Astro configuration and base-path handling
7. package.json scripts and dependencies
```

```
8. Existing test infrastructure
9. Existing arbitrary-precision dependency
10. Current CU-Time page and component responsibilities
11. Current constants inventory
12. Existing client-script conventions
13. Deployment workflow trigger branch
14. Production build status
15. Differences from this manual

Do not install packages.
Do not create or edit files.
Do not switch branches.
Do not commit, tag, push, merge, or deploy.
Return a structured verification report only.
```

2 Part 2 - Scope and Functional Parity

2.1 Migration decision

The approved migration is **Approach B**: retain the standalone HTML converter unchanged as a reference and extract approved behavior into pure TypeScript modules. The Astro website provides the integrated presentation and browser interaction.

Parity category	Requirement
Mathematical behavior	Required to match approved reference outputs
Decimal precision	Required; exact literals must enter Decimal from strings
Forward conversion	Required
Reverse conversion	Required
CE and BCE support	Required after calendar specification approval
NASA-aligned output	Required
Years-since-Big-Bang output	Required
Essential validation	Required
Copy and clear controls	Required for first interactive release
React architecture	Not required
Tailwind classes	Not required
CDN dependencies	Not permitted for the calculation engine
Original visual design	Not required

2.2 First interactive release scope

Included:

- Gregorian date and time to CU-Time.
- CU-Time to Gregorian date and time.
- CE/BCE selection and documented astronomical-year mapping.

- NASA-aligned CU-Time.
- Years since Big Bang as defined by the approved converter model.
- Full numeric and scientific-notation results.
- Accessible validation and result announcements.
- Copy and clear controls.
- Methodology and classification disclosure.

Deferred until core parity is approved:

- CSV upload and export.
- XLSX support.
- Embedded Markdown research and About tabs.
- CU-Time arithmetic calculator.
- Theme switching.
- Spreadsheet editing.
- Additional cosmological phase calculations.

2.3 Reference behavior inventory prompt

Copy-and-Paste Prompt: Audit the Reference Converter Without Editing

```
Read the complete reference file:
cosmic_breath_time_converter_v3_0_1.html

Do not modify it.

Produce a behavioral inventory with these sections:

1. External runtime dependencies and versions
2. Decimal precision and rounding configuration
3. Constants used directly by conversion functions
4. Constants mentioned only in research text
5. Input fields and validation behavior
6. Gregorian-to-CU conversion path
7. CU-to-Gregorian conversion path
8. NASA-aligned calculations
9. Formatting functions
10. Metadata calculations
11. Result fields and labels
12. Calculator behavior
13. CSV/XLSX behavior
```

```
14. Accessibility behavior
15. Potential ambiguities or defects
16. Features required for first-release parity
17. Features safe to defer

For every potential defect, label it REVIEW REQUIRED. Do not
silently propose a corrected expected value.

Do not edit repository files.
Do not install packages.
Do not commit, tag, push, merge, or deploy.
```

2.4 Parity decision record

Every behavior must receive one decision:

Permitted parity decisions

```
PRESERVE EXACTLY
PRESERVE WITH DOCUMENTED REPRESENTATION CHANGE
CORRECT AFTER APPROVAL
DEFER FROM FIRST RELEASE
REMOVE AFTER APPROVAL
OPEN QUESTION
```

Checkpoint

No engine implementation begins until the reference behavior inventory and parity decision record are reviewed and accepted.

3 Part 3 - Mathematical Constants

3.1 Direct reference-engine constants

The standalone converter constructs these values with Decimal strings and sets precision to 60 digits with half-up rounding.

Name	Exact source literal	Initial status
ANCHOR_YEAR	2000	Review
ANCHOR_JDN	2451544.5	Review
DAYS_PER_YEAR	365.2421897	Review
BASE_CU_NASA	3094213000000	Review
OFFSET	78955076.49024643522707769749119	Precision critical
NASA_UNIVERSE_AGE	13786999981.453	Review
BIG_BANG_CU_NASA	Derived by subtraction	Review

3.2 Current website constants inventory

The existing `website/src/lib/cuTime/constants.ts` also lists these CU framework values:

Additional constants currently inventoried

```
baseCu
subZtomCu
secondsPerDay
secondsPerYear
cosmicLifespan
convergenceYear
ctomStart
ztomProximity
```

They require separate provenance review because not all are used by the reference engine's core conversion functions.

Do Not Continue Until Resolved

Long decimal constants must not remain ordinary JavaScript number literals in the approved engine. Binary floating-point evaluation can discard source digits before Decimal receives the value. Store exact values as strings and construct Decimal instances from those strings.

3.3 Proposed precision-safe representation

Illustrative constants representation - not yet authorized code

```
export const CU_TIME_CONSTANT_LITERALS = {
  anchorYear: '2000',
  anchorJdn: '2451544.5',
  daysPerYear: '365.2421897',
  baseCuNasa: '3094213000000',
  offset: '78955076.49024643522707769749119',
  nasaUniverseAge: '13786999981.453',
} as const;
```

3.4 Constant-review worksheet

For each constant, record:

Field	Required entry
Name	Exact code identifier
Source literal	Character-for-character source value
Units	Years, days, JDN, CU-Time, seconds, or dimensionless
Meaning	What the constant represents
Provenance	Source file and research reference
Used by	Exact functions or equations
Precision	Required significant and decimal digits

Uncertainty	Empirical, model-defined, or not specified
Approval	Approved, provisional, rejected, or open
Change control	Whether parity expectation changes

3.5 Constant audit prompt

Copy-and-Paste Prompt: Create the Constant Registry

```
Create a proposed CU-Time constant registry from:

- cosmic_breath_time_converter_v3_0_1.html
- website/src/lib/cuTime/constants.ts
- approved CU-Time research references

Do not edit files.

Separate constants into:

A. Direct conversion-engine constants
B. Derived constants
C. Research-only constants
D. UI-only values
E. Unresolved or conflicting constants

For every constant, document:

- exact identifier
- exact literal as text
- units
- meaning
- source location
- functions that use it
- precision requirement
- uncertainty or model status
- whether it is approved for the first engine

Detect values that would lose precision as JavaScript numbers.
Do not change values.
Mark discrepancies as REVIEW REQUIRED.
```

Checkpoint

Part 3 is complete only after the direct engine constants are approved as exact strings, derived values have explicit formulas, and research-only values are excluded from the first conversion engine unless separately authorized.

4 Part 4 - Calendar and Time Specification

4.1 Reference calendar behavior

The reference converter uses a proleptic Gregorian-style Julian Day Number algorithm. It maps BCE input to astronomical years with:

$$Y_{\text{astronomical}} = 1 - Y_{\text{BCE}}.$$

Therefore, 1 BCE maps to astronomical year 0, and 2 BCE maps to astronomical year -1.

The anchor JDN is `2451544.5`. The implementation treats the input time as UTC-like civil time and adds a day fraction based on 86,400 seconds.

4.2 Specification decisions required

- Confirm the proleptic Gregorian calendar across all supported years.
- Confirm whether year 0 is accepted as direct CE input or only used internally.
- Confirm BCE display and astronomical-year conversion.
- Confirm the midnight/noon interpretation of the JDN anchor.
- Confirm UTC as the only supported time basis.
- Define valid month/day combinations and leap-year rules.
- Define maximum supported year and safe conversion limits.
- Define second precision and fractional-second policy.
- Define rollover behavior when rounded seconds reach 60.

4.3 Gregorian leap-year rule

For the proleptic Gregorian calendar, a year is a leap year when it is divisible by 4, except years divisible by 100 are not leap years unless they are divisible by 400. BCE application must use the approved astronomical-year mapping consistently.

4.4 Reference JDN formula

For year Y, month M, day D, and time values h, m, and s:

$$
\begin{aligned}
a &= \left\lfloor \frac{14 - M}{12} \right\rfloor, \\
y &= Y + 4800 - a, \\
m' &= M + 12a - 3, \\
J_{\text{int}} &= D + \left\lfloor \frac{153m' + 2}{5} \right\rfloor + 365y + \left\lfloor \frac{y}{4} \right\rfloor - \left\lfloor \frac{y}{100} \right\rfloor + \left\lfloor \frac{y}{400} \right\rfloor - 32045, \\
f &= \frac{3600h + 60m + s}{86400}, \\
J &= J_{\text{int}} - 0.5 + f.
\end{aligned}
$$

This records the reference algorithm. It is not a scientific endorsement of all supported date extrapolations.

4.5 Known validation gap

The reference input validation checks ranges such as month 1–12 and day 1–31, but does not fully reject invalid combinations such as April 31 or a non-leap-year February 29 before conversion.

Do Not Continue Until Resolved

Improved date validation is a desirable correction, but it changes observable behavior. Record it as a separately approved correction with explicit tests.

4.6 Calendar specification prompt

Copy-and-Paste Prompt: Draft the Calendar Specification

```
Draft the formal calendar and time specification for the integrated
converter.

Do not edit code.

Address:

1. Proleptic Gregorian calendar definition
2. Julian Day Number convention
3. Anchor JDN and its civil date/time meaning
4. UTC handling
5. CE/BCE input and output
6. Astronomical year zero mapping
7. Leap-year rules
8. Month-length validation
9. Maximum supported year
10. Seconds and fractional seconds
11. Rounding and day rollover
12. Invalid-input behavior
13. Round-trip tolerance
14. Known differences between the reference behavior and the
```

```
    proposed validated behavior

Label each proposed behavioral change REVIEW REQUIRED.
Do not silently approve corrections.
```

5 Part 5 - Gregorian-to-CU Conversion

5.1 Reference conversion path

The approved path to be specified and tested is:

Forward conversion sequence

```
Gregorian date, time, and era
  -> astronomical year
  -> Julian Day Number
  -> delta days from ANCHOR_JDN
  -> delta years using DAYS_PER_YEAR
  -> NASA-aligned CU-Time
  -> subtract OFFSET
  -> converter-aligned CU-Time
  -> derived and formatted outputs
```

5.2 Reference equations

Let J be the calculated JDN and J_0 the anchor JDN.

$$\begin{aligned}
\Delta J &= J - J_0, \\
\Delta Y &= \frac{\Delta J}{D_Y}, \\
CU_{\text{NASA}} &= CU_{\text{NASA},0} + \Delta Y, \\
CU_{\text{converter}} &= CU_{\text{NASA}} - O, \\
Y_{\text{BigBang}} &= Y_{\text{NASA},0} + \left(CU_{\text{NASA}} - CU_{\text{NASA},0}\right),
\end{aligned}$$

where D_Y is `DAYS_PER_YEAR` and O is `OFFSET`.

5.3 Required forward output fields

- Original input date, era, and time.
- Normalized Gregorian date.
- Converter-aligned CU-Time formatted by magnitude.
- Full converter-aligned numeric value.

- Scientific notation.
- NASA-aligned CU-Time formatted by magnitude.
- Full NASA-aligned numeric value.
- NASA-aligned scientific notation.
- Years since Big Bang according to the converter model.
- Approved metadata fields.

5.4 Potential formatting ambiguity

The reference `formatCUTime` function appears to combine a remainder that may already contain the fractional component with `cuTime.mod(1)`. This may double-count part of the fraction. The behavior must be captured in a golden vector before any correction is considered.

5.5 Forward specification prompt

Copy-and-Paste Prompt: Specify Gregorian-to-CU Conversion

```
Using the approved constants registry and calendar specification,
draft the normative Gregorian-to-CU conversion specification.

Do not implement code.

Include:

- typed inputs
- normalization rules
- validation order
- exact Decimal operations
- forward equations
- rounding points
- result object fields
- display formatting
- error conditions
- test invariants
- reference-converter comparison procedure

Identify any reference behavior that appears ambiguous or defective.
Label it REVIEW REQUIRED and preserve the original expected output
until an explicit correction is approved.
```

Checkpoint

The forward algorithm is authorized for implementation only after its inputs, equations, output schema, rounding behavior, and golden expectations are approved.

6 Part 6 - CU-to-Gregorian Conversion

6.1 Reference reverse path

Reverse conversion sequence

```
Converter-aligned CU-Time
  -> add OFFSET
  -> NASA-aligned CU-Time
  -> delta years from BASE_CU_NASA
  -> Julian Day Number
  -> Gregorian calendar conversion
  -> CE/BCE display
  -> derived and formatted outputs
```

6.2 Reference reverse equations

$$\begin{aligned} CU_{\mathrm{NASA}} &= CU_{\mathrm{converter}} + O, \\ \Delta Y &= CU_{\mathrm{NASA}} - CU_{\mathrm{NASA},0}, \\ J &= J_0 + \Delta Y D_Y. \end{aligned}$$

The JDN is then converted to Gregorian components using the reference `jdnToDate` algorithm.

6.3 Reverse-conversion review points

- Decimal input syntax and whitespace handling.
- Negative CU-Time support.
- Maximum supported magnitude.
- Gregorian year conversion from Decimal to JavaScript number.
- Rounding seconds to the nearest integer.
- Carrying seconds to minutes and minutes to hours.
- Carrying hour 24 into the next calendar day.
- CE/BCE display convention.
- Round-trip comparison tolerance.

The reference function reduces hours from 24 to 0 but does not visibly increment the calendar date in that branch. This is a potential rollover defect and must be evaluated with a targeted vector.

6.4 Reverse specification prompt

Copy-and-Paste Prompt: Specify CU-to-Gregorian Conversion

```
Using the approved constants registry and calendar specification,
draft the normative CU-to-Gregorian conversion specification.

Do not implement code.

Include:

- accepted Decimal input grammar
- exact offset and JDN equations
- Gregorian reconstruction algorithm
- CE/BCE display rules
- rounding and carry rules
- next-day rollover behavior
- output object fields
- error handling
- maximum supported magnitude
- round-trip invariants
- reference parity comparison

Create targeted REVIEW REQUIRED cases for second, minute, hour,
day, month, year, leap-day, and BCE rollovers.
```

7 Part 7 - Golden Reference Test Vectors

7.1 Purpose

A golden vector records an input and the complete output produced by the unchanged reference converter. Golden vectors protect behavior during the TypeScript migration. They do not automatically prove that the underlying mathematics is correct.

7.2 Required vector set

ID	Input	Reason
F-01	01/01/2000 CE 00:00:00	Anchor date
F-02	06/05/2025 CE 00:12:00	Metadata reference instant
F-03	06/05/2025 CE 00:00:00	Nearby reference boundary
F-04	07/19/2026 CE 00:00:00	Manual verification date
F-05	02/29/2000 CE 12:34:56	Valid leap day
F-06	02/29/1900 CE 00:00:00	Invalid Gregorian leap day
F-07	04/31/2025 CE 00:00:00	Invalid month/day combination
F-08	01/01/0001 CE 00:00:00	CE boundary
F-09	01/01/0001 BCE 00:00:00	Astronomical year zero mapping
F-10	Maximum supported year	Magnitude boundary

R-01	Anchor CU-Time	Reverse anchor
R-02	Approved present CU-Time	Reference example
R-03	Negative CU-Time	Input domain
R-04	Rollover-sensitive CU-Time	Seconds/day carry
RT-01	Forward then reverse	Round-trip invariant
RT-02	Reverse then forward	Round-trip invariant

7.3 Derived anchor candidate

At the anchor JDN, the reference equations give:

$$CU_{\text{converter},0} = 3094213000000 - 78955076.4902464352270776974911 9.$$

This produces the source literal represented in the current website inventory as approximately `3094134044923.509753564772922302508810`. It must still be captured from the running reference converter and compared character by character before becoming a golden value.

7.4 Golden vector record format

Required vector record

```
ID:
Reference converter version:
Reference file hash:
Input mode:
Input date/time/era or CU-Time:
Observed output labels:
Observed full numeric values:
Observed formatted values:
Observed scientific notation:
Observed metadata:
Browser and version:
Capture date:
Reviewer:
Parity decision:
Notes and open questions:
```

7.5 Reference hash command

Record the unchanged reference file hash

```
shasum -a 256 cosmic_breath_time_converter_v3_0_1 html
```

Use the actual repository path when the file is stored elsewhere.

7.6 Golden vector prompt

Copy-and-Paste Prompt: Prepare the Golden Vector Plan

```
Create the golden reference vector worksheet for the unchanged
HTML converter.

Do not edit the reference converter or website code.

For each required vector:

- define exact input
- define which output fields must be captured
- identify the behavior being tested
- identify expected validation behavior
- identify whether the case is parity-only or correction-review
- define a reproducible capture procedure

Do not invent outputs that have not been observed from the reference
converter. Leave observed-output fields blank until captured.
```

7.7 Golden vector approval gate

☐ Reference file hash recorded.

☐ All required forward vectors captured.

☐ All required reverse vectors captured.

☐ Invalid-date behavior captured.

☐ BCE behavior captured.

☐ Rollover behavior captured.

☐ Full precision copied without spreadsheet conversion.

☐ Each ambiguity has a review decision.

☐ Golden vector dataset approved before engine coding.

8 Part 8 - TypeScript Conversion Engine Architecture

8.1 Approved architecture direction

The engine should contain pure modules with no direct DOM access. The Astro component and browser controller should consume the engine through typed interfaces.

Target CU-Time architecture

```
website/src/lib/cuTime/
|-- constants.ts
|-- types.ts
|-- decimal.ts
|-- validation.ts
|-- calendar.ts
|-- converter.ts
|-- format.ts
|-- metadata.ts
`-- index.ts

website/src/components/
`-- CUTimeConverter.astro

website/src/scripts/
`-- cu-time-converter.ts

website/src/pages/
`-- cu-time.astro

website/src/lib/cuTime/__tests__/
|-- constants.test.ts
|-- calendar.test.ts
|-- converter.test.ts
|-- format.test.ts
`-- golden-vectors.test.ts
```

8.2 Module responsibilities

Module	Responsibility
`constants.ts`	Exact approved literal registry; no calculations
`types.ts`	Public inputs, outputs, errors, era, and vector types
`decimal.ts`	Decimal configuration and safe constructors
`validation.ts`	Date, time, era, numeric, and domain validation
`calendar.ts`	Gregorian/JDN conversion only
`converter.ts`	Forward, reverse, NASA, and Big-Bang calculations
`format.ts`	Numeric and magnitude presentation
`metadata.ts`	Approved secondary result calculations
`index.ts`	Controlled public exports
Browser controller	Form events, state, rendering, copy, and clear
Astro component	Semantic form, results, labels, status, and styles

8.3 Dependency plan

No arbitrary-precision dependency or test framework is currently installed. After specification approval, the intended dependency additions are:

Install approved dependencies - run only when authorized

```
cd "$(git rev-parse --show-toplevel)/website" || exit 1

npm install decimal.js
npm install --save-dev vitest
```

The exact package versions written to `package-lock.json` become part of the milestone record.

8.4 Required package scripts

Proposed package.json scripts

```
"test": "vitest run",
"test:watch": "vitest",
"check": "astro check"
```

If `astro check` is added, the compatible checker package must also be installed according to the then-current Astro requirements.

8.5 Type and error design

The engine should return structured data and typed errors rather than formatted HTML strings. Suggested categories include:

Suggested error codes

```
INVALID_DATE_FORMAT
INVALID_TIME_FORMAT
INVALID_MONTH
INVALID_DAY
INVALID_LEAP_DAY
INVALID_ERA
INVALID_CU_TIME
OUT_OF_SUPPORTED_RANGE
DIVISION_BY_ZERO
PRECISION_LIMIT
INTERNAL_CONVERSION_ERROR
```

8.6 Architecture planning prompt

Copy-and-Paste Prompt: Finalize the TypeScript Architecture

```
Using the approved specification and verified Astro repository,
finalize the TypeScript engine architecture.

Do not edit files.
```

```
For every proposed file, define:

- responsibility
- imports
- exports
- public types
- error behavior
- Decimal boundary
- test ownership
- prohibited responsibilities

Requirements:

- pure conversion functions
- exact string literals for precision-critical constants
- no DOM access in lib/cuTime
- no React
- no Tailwind dependency
- no calculation CDN
- lightweight browser controller
- compatibility with Astro strict TypeScript
- compatibility with the configured base path
- behavior controlled by golden vectors

Return the file plan and dependency plan only.
```

Checkpoint

The architecture is complete only when each responsibility has one owner, calculation code is independent of the UI, and no precision-critical value enters Decimal through a JavaScript number.

9 Part 9 - Automated Testing

9.1 Testing layers

1. **Constant tests**: exact literals and derived identities.
2. **Calendar tests**: JDN, leap years, BCE, and rollovers.
3. **Conversion tests**: forward and reverse equations.
4. **Golden tests**: complete parity with captured reference outputs.
5. **Round-trip tests**: input recovery within approved tolerance.
6. **Validation tests**: malformed and impossible inputs.
7. **Formatting tests**: magnitude labels and full numeric output.

8. **Build test**: Astro static generation.
9. **Manual accessibility test**: keyboard and screen-reader behavior.

9.2 Test commands

Run the automated and production checks

```
cd "$(git rev-parse --show-toplevel)/website" || exit 1

npm test
npm run build
```

9.3 Numeric comparison policy

Use exact string equality when the reference output is specified as exact and both implementations use the same approved Decimal operations. Use an explicit Decimal tolerance only when the specification documents a rounding boundary or conversion loss. Never use an undocumented floating-point epsilon.

9.4 Minimum test requirements

- ☐ Anchor JDN conversion.
- ☐ Anchor CU-Time conversion.
- ☐ All golden vectors.
- ☐ CE/BCE boundary.
- ☐ Leap-year and invalid leap-day behavior.
- ☐ Month-length validation.
- ☐ Midnight and near-midnight rollover.
- ☐ Negative and large CU-Time inputs.
- ☐ Invalid numeric syntax.
- ☐ Full precision literal preservation.
- ☐ Format behavior, including the fractional remainder review.
- ☐ Metadata reference instant and units.
- ☐ Round-trip invariants.

9.5 Test implementation prompt

Copy-and-Paste Prompt: Implement the Approved Test Harness

```
Implement only the approved automated test foundation and test
vectors.

Preconditions:

- active branch is feature/cu-time-converter
- working tree is understood
- constants registry is approved
- calendar specification is approved
- golden vectors are approved

Requirements:

- install only approved dependencies
- configure Vitest minimally
- add test scripts
- store golden vectors in a reviewable text-based format
- test exact strings where required
- use documented Decimal tolerances only where approved
- do not implement the interactive UI
- do not modify the reference HTML converter
- run npm test
- run npm run build
- report every changed file and dependency

Do not commit, tag, push, merge, or deploy.
```

9.6 Testing completion gate

Successful Result

The engine may be connected to the page only after all approved unit and golden vector tests pass and the production build succeeds.

10 Part 10 - Interactive Astro Page Integration

10.1 Integration objective

Replace the current “Interactive converter under development” content with an accessible working interface while preserving the page’s documentation, classification, research-status language, and visual system.

10.2 First-release interface

- Mode selector or clearly separated forward and reverse forms.

- Date input with explicit `MM/DD/YYYY` help.
- Time input with explicit 24-hour `HH:MM:SS` help.
- CE/BCE selector.
- CU-Time decimal input.
- Convert controls.
- Clear controls.
- Copy result control.
- Validation summary and field-level errors.
- Result region announced with `aria-live`.
- Full precision result preserved as text.
- Scientific and formatted display values.
- Calculation-method disclosure.

10.3 Astro client strategy

The project currently contains no detected client scripts and no UI framework. The preferred first implementation is:

Client strategy

```
Astro semantic markup
  + processed TypeScript <script>
  + imported pure CU-Time modules
  + DOM event listeners scoped to the component
  + no React island
  + no browser-side Babel
  + no Tailwind CDN
```

The browser controller should progressively enhance valid HTML. All calculation logic remains in `website/src/lib/cuTime/`.

10.4 Route and base-path requirements

The source page remains:

Source and route

```
Source: website/src/pages/cu-time.astro
Development/public route:
/CosmicUniversalismStatement/cu-time/
Generated static file:
```

```
website/dist/cu-time/index.html
```

Use the existing centralized base-path handling for internal links and assets. The calculator itself should not hard-code the repository base into mathematical logic.

10.5 Accessibility requirements

- Every field has a visible label.
- Format help is associated with the field.
- Errors identify the field and corrective action.
- Focus moves only when necessary and predictably.
- Results are announced without stealing keyboard focus.
- Copy buttons report success in text.
- Color is never the only status indicator.
- Controls remain usable at 200% zoom.
- Mobile input controls do not overflow.
- Motion is unnecessary for converter operation.

10.6 UI planning prompt

Copy-and-Paste Prompt: Plan the Interactive CU-Time Interface

```
Using the approved engine API and the existing CU-Time page design,
create the implementation specification for the first interactive
converter release.

Do not edit files.

Specify:

1. Semantic form structure
2. Forward and reverse modes
3. Exact field labels and help text
4. Validation presentation
5. Result region and fields
6. Copy and clear behavior
7. aria-live behavior
8. Keyboard behavior
9. Responsive layout
10. Astro script loading and module imports
11. Error-state styling using existing tokens
12. Research classification and methodology disclosure
```

```
13. Deferred features
14. Manual test procedure
15. Acceptance criteria

Do not add React, Tailwind, or CDN calculation dependencies.
```

10.7 UI implementation prompt

Copy-and-Paste Prompt: Integrate the Approved Converter with Astro

```
Implement only the approved first-release interactive CU-Time
interface.

Preconditions:

- active branch is feature/cu-time-converter
- engine tests pass
- golden parity tests pass
- npm run build succeeds before UI changes

Requirements:

- update CUTimeConverter.astro
- connect the approved pure TypeScript engine
- use lightweight browser TypeScript
- preserve the surrounding cu-time.astro documentation
- preserve research classifications
- use existing design tokens
- provide visible labels and accessible errors
- announce results with aria-live
- preserve full precision as text
- include copy and clear controls
- support desktop and mobile layouts
- do not add React
- do not add Tailwind
- do not use CDN calculation libraries
- do not modify the reference HTML converter
- run npm test
- run npm run build
- report all changed files and manual test results

Do not commit, tag, push, merge, or deploy.
```

10.8 Local manual test

Run the integrated page locally

```
cd "$(git rev parse --show toplevel)/website" || exit 1

npm run dev
```

Open:

Local CU-Time route

```
http://localhost:4321/CosmicUniversalismStatement/cu-time/
```

Test keyboard navigation, mobile width, all golden inputs, invalid inputs, copy, clear, and browser-console output.

11 Part 11 - Acceptance, Review, Milestones, and Merge

11.1 Parity report

Before merge, create a report that maps every approved reference vector to the new engine result.

ID	Reference	TypeScript	Result	Decision
			Pass/Fail	
			Pass/Fail	
			Pass/Fail	
			Pass/Fail	

11.2 Final acceptance checklist

☐ Active work remained on `feature/cu-time-converter`.

☐ `main` was not modified.

☐ Original HTML reference converter is unchanged.

☐ Reference hash matches the recorded hash.

☐ Constants registry is approved.

☐ Calendar specification is approved.

☐ Forward specification is approved.

☐ Reverse specification is approved.

☐ Golden vectors are approved.

☐ Decimal precision is preserved from strings.

- ☐ Automated tests pass.
- ☐ Golden parity tests pass.
- ☐ Round-trip tests pass within documented tolerance.
- ☐ Invalid dates are handled as approved.
- ☐ BCE behavior is documented and tested.
- ☐ Rollover behavior is documented and tested.
- ☐ Converter is interactive at `/CosmicUniversalismStatement/cu-time/`.
- ☐ Keyboard and mobile testing pass.
- ☐ No console errors remain.
- ☐ `npm run build` succeeds.
- ☐ Working tree contains only understood files.
- ☐ Documentation and current-state record are updated.
- ☐ Explicit approval to commit has been given.
- ☐ Explicit approval to merge has been given.

11.3 Required final verification commands

Final feature verification

```
cd "$(git rev-parse --show-toplevel)" || exit 1

git branch --show-current
git status --short --branch

cd website || exit 1
npm test
npm run build
cd ..

git diff --check
git status --short --branch
```

11.4 Recommended milestone sequence

Milestone	Meaning
`website-v1.1-cu-time-foundation`	Existing UI and repository foundation
`website-v1.1-cu-time-specification`	Constants, calendar, formulas, and vectors approved
`website-v1.1-cu-time-engine`	TypeScript engine and automated tests approved
`website-v1.1-cu-time-converter`	Interactive page, parity, accessibility, and build approved

Tags should be created only after explicit approval and should point to clean, reviewed commits. The final converter release tag is preferably created after the approved feature is merged into `website`.

11.5 Commit planning

Suggested logical commits, subject to the actual approved work:

Suggested commit sequence

```
Document CU-Time mathematical specification
Add precision-safe CU-Time constants registry
Add CU-Time calendar conversion module
Add CU-Time conversion engine
Add CU-Time golden vector tests
Integrate interactive CU-Time converter
Document CU-Time parity and acceptance
```

11.6 Merge procedure

1. Confirm all acceptance checks are complete.
2. Confirm the feature branch is pushed and clean.
3. Review the complete branch diff against `website`.
4. Obtain explicit approval to merge.
5. Merge `feature/cu-time-converter` into `website`.
6. Run the tests and production build on `website`.
7. Push `website` only after verification.
8. Confirm the website deployment workflow completes.
9. Verify the public CU-Time route.
10. Create the approved release tag.
11. Update both implementation manuals and the current-state record.

Do Not Continue Until Resolved

The deployment workflow is configured to run on pushes to `website`, not the feature branch. Merging and pushing to `website` may publish the converter. Treat merge approval as release approval unless the workflow is intentionally disabled first.

11.7 Final review prompt

Copy-and-Paste Prompt: Final CU-Time Converter Review

```
Perform a final no-edit review of the CU-Time converter feature.

Compare feature/cu-time-converter against website.

Verify:

1. Branch and working-tree state
2. Original reference converter hash
3. Constants and precision representation
4. Calendar specification compliance
5. Forward and reverse specification compliance
6. Golden vector parity
7. Automated test results
8. Round-trip results
9. Validation behavior
10. Accessibility and responsive behavior
11. Target route and Astro base-path behavior
12. Dependency changes
13. Files outside the authorized scope
14. Build success
15. Documentation updates
16. Deployment implications
17. Remaining open questions

Report findings by severity.
Do not edit, commit, tag, push, merge, or deploy.
```

11.8 Final controlled statement

Controlled Migration Rule

The Cosmic Breath Time Converter must be migrated by preserving observable reference behavior, documenting constants and calendar assumptions, capturing golden vectors, implementing pure precision-safe TypeScript functions, proving parity through automated tests, and only then connecting the engine to the Astro interface.

The original HTML file remains unchanged during migration. Potential mathematical or implementation corrections are documented and approved independently; they are never introduced silently under the label of parity.

A Appendix A - Current-State Record Template

After implementation begins, maintain a repository record such as:

website/docs/cu-time/CU_TIME_CURRENT_STATE.md

```
# CU-Time Current State

Repository: willmaddock/CosmicUniversalismStatement
Protected branch: main
Stable website branch: website
Active feature branch: feature/cu-time-converter
Current HEAD: [UPDATE]
Current milestone: [UPDATE]
Current tags: [UPDATE]
Target route: /CosmicUniversalismStatement/cu-time/
Reference converter hash: [UPDATE]

## Completed
- [UPDATE]

## In Progress
- [UPDATE]

## Pending
- [UPDATE]

## Tests
- npm test: [RESULT]
- npm run build: [RESULT]

## Open Questions
- [UPDATE]

## Merge Status
- [UPDATE]
```

B Appendix B - Phase Authorization Prompt

Use this prompt before every implementation phase.

Copy-and-Paste Prompt: Authorize One Controlled Phase

```
We are authorizing one controlled CU-Time implementation phase.

Authorized phase:
[INSERT PHASE]

Approved specification:
[INSERT OR REFERENCE APPROVED SECTION]
```

```
Before editing:

1. Confirm branch feature/cu-time-converter.
2. Confirm working-tree status.
3. List files expected to change.
4. List dependencies expected to change.
5. State the tests that must pass.
6. State the completion gate.

During implementation:

- change only the authorized scope
- preserve the reference HTML file
- preserve exact Decimal literals
- do not silently correct behavior
- document all deviations

After implementation:

- run the authorized tests
- run npm run build
- report every changed file
- report warnings, failures, and open questions

Do not commit, tag, push, merge, or deploy.
```

C Appendix C - Manual Revision Record

Version	Date	Change
1.0	July 19, 2026	Initial controlled converter migration manual based on the verified CU-Time foundation branch, repository inspection, and successful Astro production build.

End of Cosmic Breath Time Converter Implementation Manual